What is Data?

Data is a collection of a distinct small unit of information. It can be used in a variety of forms like text, numbers, media, bytes, etc. it can be stored in pieces of paper or electronic memory, etc.

Word 'Data' is originated from the word 'datum' that means 'single piece of information.' It is plural of the word datum.

In computing, Data is information that can be translated into a form for efficient movement and processing. Data is interchangeable.

What is Database?

A **database** is an organized collection of data, so that it can be easily accessed and managed. The **main purpose** of the database is to operate a large amount of information by storing, retrieving, and managing data.

There are many **databases available** like MySQL, Sybase, Oracle, MongoDB, Informix, PostgreSQL, SQL Server, etc. Modern databases are managed by the database management system (DBMS).

SQL or Structured Query Language is used to operate on the data stored in a database. SQL depends on relational algebra and tuple relational calculus.

Database Management System

The database is a collection of database and the software to manage that database. It is also used to organize the data in the form of a table, schema, views, and reports, etc. **For example:** The college Database organizes the data about the admin, staff, students and faculty etc.

Using the database, you can easily retrieve, insert, and delete the information.

A DBMS is software designed to manage, organize, and manipulate data efficiently. It acts as a central hub for data storage, retrieval, and analysis, empowering various users like analysts, developers, and even casual users to interact with the information effectively.

- Database management system is a software which is used to manage the database. For example: MySQL, Oracle, etc are a very popular commercial database which is used in different applications.

- DBMS provides an interface to perform various operations like database creation, storing data in it, updating data, creating a table in the database and a lot more.
- It provides protection and security to the database. In the case of multiple users, it also maintains data consistency.

- DBMS contains information about a particular enterprise
    - Collection of interrelated data
    - Set of programs to access the data
    - An environment that is both convenient and efficient to use

Why Use a DBMS?

Using a DBMS offers several advantages over traditional, file-based data management:

- **Data Efficiency:** Organizes data in structured formats like tables, minimizing redundancy and maximizing storage space.
- **Data Integrity:** Enforces data consistency and accuracy through rules and constraints, reducing errors and anomalies.
- **Concurrent Access:** Multiple users can access and modify data simultaneously with efficient control mechanisms.
- **Security:** Provides robust security features to protect sensitive data from unauthorized access and modifications.
- **Backup and Recovery:** Offers data backup and recovery mechanisms to minimize downtime and data loss in case of failures.
- **Data Analysis:** Facilitates data analysis and reporting through query languages and integrated tools.

DBMS allows users the following tasks:

- **Data Definition:** It is used for creation, modification, and removal of definition that defines the organization of data in the database.
- **Data Updation:** It is used for the insertion, modification, and deletion of the actual data in the database.
- **Data Retrieval:** It is used to retrieve the data from the database which can be used by applications for various purposes.
- **User Administration:** It is used for registering and monitoring users, maintain data integrity, enforcing data security, dealing with concurrency control, monitoring performance and recovering information corrupted by unexpected failure.

Characteristics of DBMS

- It uses a digital repository established on a server to store and manage the information.
- It can provide a clear and logical view of the process that manipulates data.
- DBMS contains automatic backup and recovery procedures.
- It contains ACID properties which maintain data in a healthy state in case of failure.
- It can reduce the complex relationship between data.
- It is used to support manipulation and processing of data.
- It is used to provide security of data.
- It can view the database from different viewpoints according to the requirements of the user.

DATABASE APPLICATIONS EXAMPLES

- Enterprise Information
  - Sales: customers, products, purchases
  - Accounting: payments, receipts, assets
  - Human Resources: Information about employees, salaries, payroll taxes.
- Manufacturing: management of production, inventory, orders, supply chain.
- Universities: registration, grades
- Banking and finance
  - Customer information, accounts, loans, and banking transactions.
  - Credit card transactions
  - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data
- Airlines: reservations, schedules

- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
    - Online retailers: order tracking, customized recommendations
    - Online advertisements
- Document databases
- Navigation systems: For maintaining the locations of varies places of interest along with the exact routes of roads, train systems, buses, etc.

Advantages of DBMS

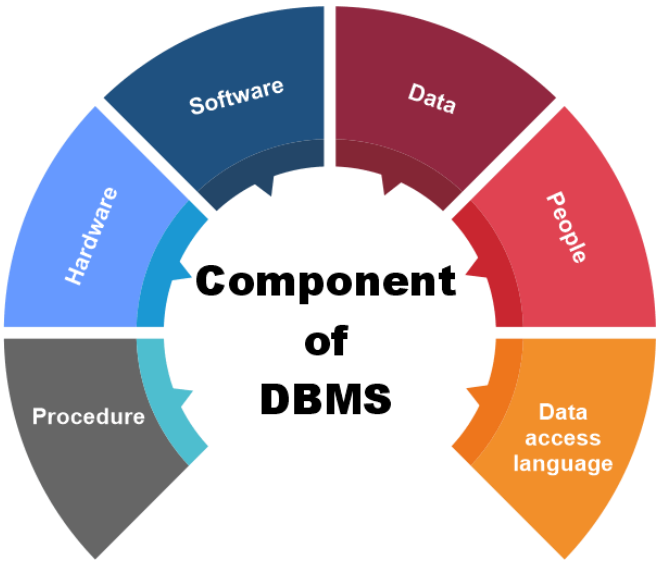
- **Controls database redundancy:** It can control data redundancy because it stores all the data in one single database file and that recorded data is placed in the database.
- **Data sharing:** In DBMS, the authorized users of an organization can share the data among multiple users.
- **Easily Maintenance:** It can be easily maintainable due to the centralized nature of the database system.
- **Reduce time:** It reduces development time and maintenance need.
- **Backup:** It provides backup and recovery subsystems which create automatic backup of data from hardware and software failures and restores the data if required.
- **multiple user interface:** It provides different types of user interfaces like graphical user interfaces, application program interfaces

Disadvantages of DBMS

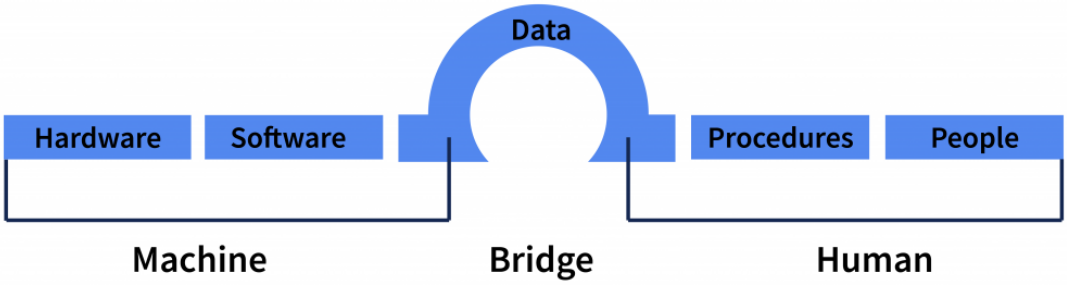
- **Cost of Hardware and Software:** It requires a high speed of data processor and large memory size to run DBMS software.
- **Size:** It occupies a large space of disks and large memory to run them efficiently.
- **Complexity:** Database system creates additional complexity and requirements.
- **Higher impact of failure:** Failure is highly impacted the database because in most of the organization, all the data stored in a single database and if the database is damaged due to electric failure or database corruption then the data may be lost forever.

Key Components of a DBMS:

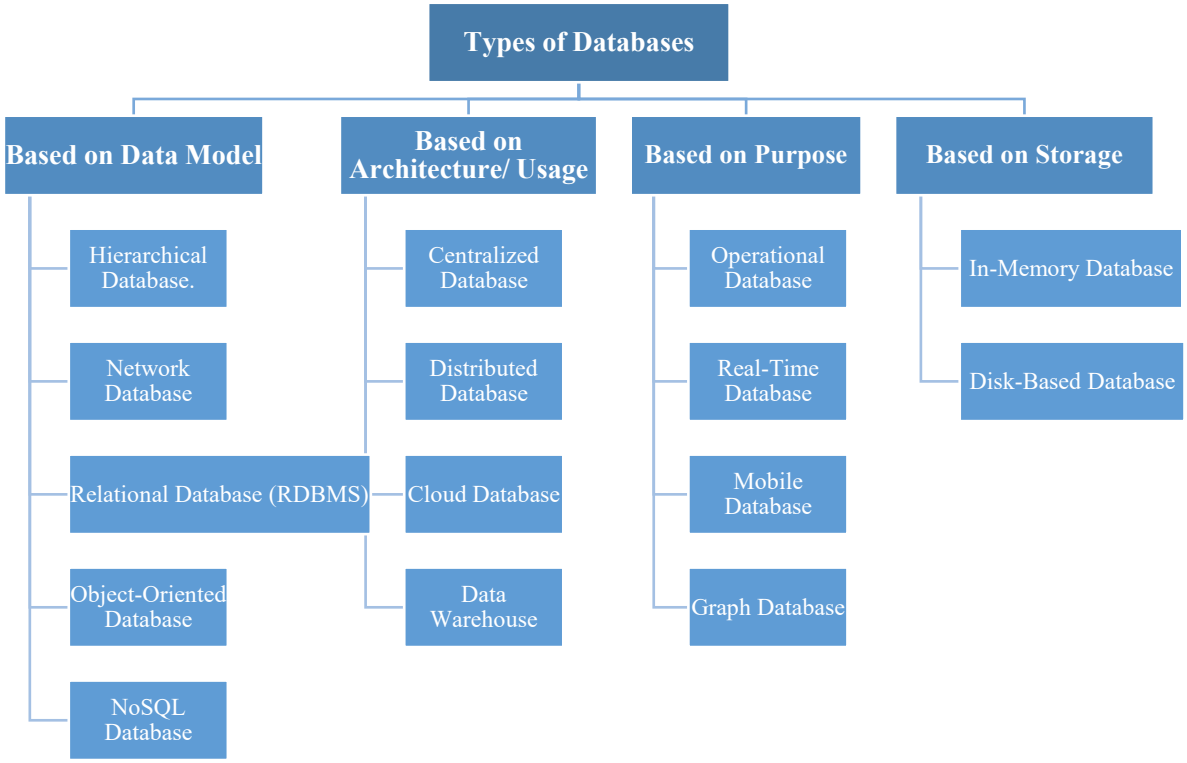
- **Hardware:** Physical elements like storage devices, computers, and network equipment that provide the raw processing power and capacity.
- **Software:** DBMS software programs responsible for data storage, retrieval, manipulation, and security, along with the operating system and network software for communication and resource management.



- **Data:** The actual information stored in the database, often organized in tables with relationships defined by columns and rows.
- **Database Access Language (DAL):** A language like SQL used to interact with the database, write queries to retrieve or manipulate data.
- **Procedures:** Predefined instructions for managing the database, including backup routines, security mechanisms, and data integrity checks.
- **Users:** Individuals who interact with the database, including database administrators, application developers, analysts, and end users.
- **Metadata:** Information about the data itself, like table structure, data types, ownership, and access control rules.



Types of Databases

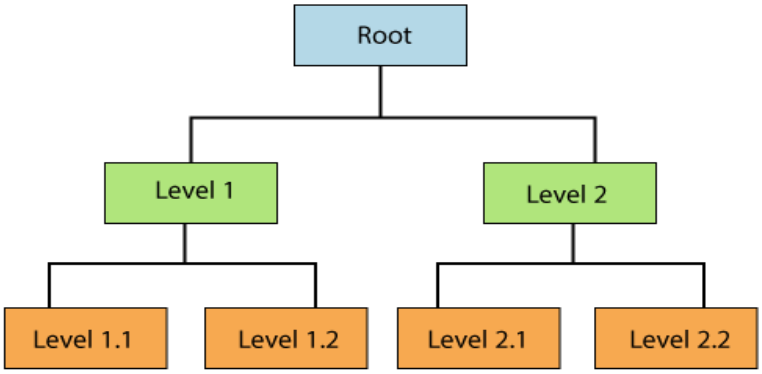


1. Based on Data Model

a) Hierarchical Database

- Data organized in a **tree structure** (parent–child). One-to-many relationship. *Example:* IBM IMS.

- It is the type of database that stores data in the form of parent-children relationship nodes. Here, it organizes data in a tree-like structure.
- Data get stored in the form of records that are connected via links. Each child record in the tree will contain only one parent. On the other hand, each parent record can have multiple child records.



Hierarchical Database

b) Network Database

- Data organized as a **graph** and it supports many-to-many relationships. *Example:* IDMS.
- It is the database that typically follows the network data model. Here, the representation of data is in the form of nodes connected via links between them. Unlike the hierarchical database, it allows each record to have multiple children and parent nodes to form a generalized graph structure.

c) Relational Database (RDBMS)

- Data stored in **tables (rows & columns)**. It uses **SQL** for queries. *Examples:* MySQL, Oracle, PostgreSQL.
- This database is based on the relational data model, which stores data in the form of rows(tuple) and columns(attributes), and together forms a table(relation). A relational database uses SQL for storing, manipulating, as well as maintaining the data. E.F. Codd invented the database in 1970. Each table in the database carries a key that makes the data unique from others. **Examples** of Relational databases are MySQL, Microsoft SQL Server, Oracle, etc.

d) Object-Oriented Database

- Stores data as **objects** (data + methods). It is used in complex applications. *Example:* ObjectDB.
- The type of database that uses the object-based data model approach for storing data in the database system. The data is represented and stored as objects which are similar to the objects used in the object-oriented programming language.

e) NoSQL Database

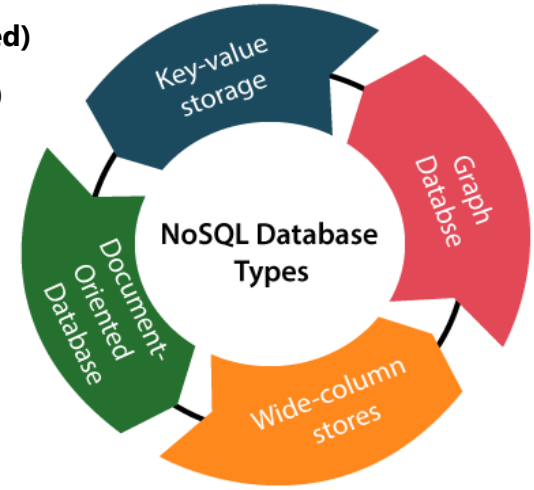
- Designed for **big data and scalability**.

Non-SQL/Not Only SQL is a type of database that is used for storing a wide range of data sets. It is not a relational database as it stores data not only in tabular form but in several different ways. It came into existence when the demand for building modern applications increased. Thus, NoSQL presented a wide variety of database technologies in response to the demands.

Types: Key-Value, Document, Column-Family, Graph.

Some examples of NoSQL database system with their category are:

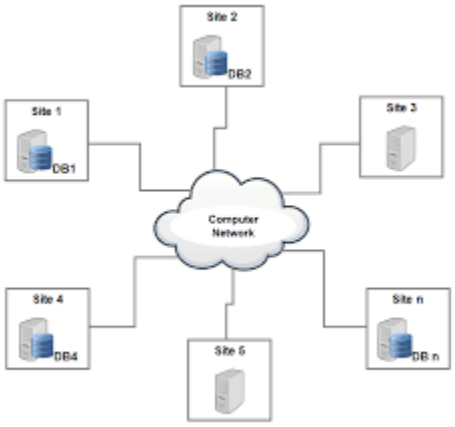
- MongoDB, CouchDB, Cloudant **(Document-based)**
- Memcached, Redis, Coherence **(key-value store)**
- HBase, Big Table, Accumulo **(Tabular)**



Advantages of NoSQL Database

- It enables good productivity in the application development as it is not required to store data in a structured format.
- It is a better option for managing and handling large data sets.
- It provides high scalability.
- Users can quickly access data from the database through key-value.

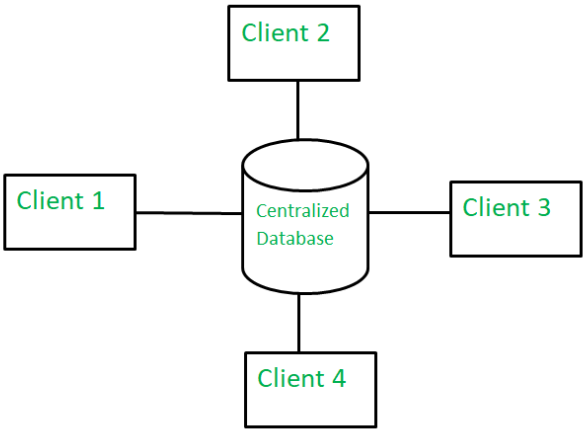
2. Based on Architecture / Usage



a) Centralized Database

- Stored at a **single location**. And easy management but single point of failure.

It is the type of database that stores data at a centralized database system. It comforts the users to access the stored data from different locations through several applications. These applications contain the authentication process to let users access data securely. An example of a Centralized database can be Central Library that carries a central database of each library in a college/university.



Advantages of Centralized Database

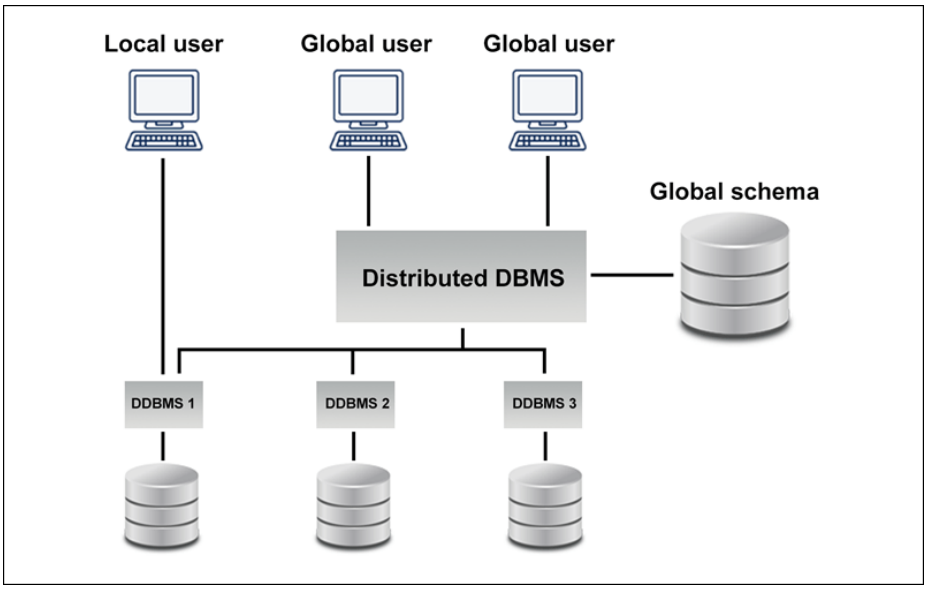
- It has decreased the risk of data management, i.e., manipulation of data will not affect the core data.
- Data consistency is maintained as it manages data in a central repository.
- It provides better data quality, which enables organizations to establish data standards.
- It is less costly because fewer vendors are required to handle the data sets.

Disadvantages of Centralized Database

- The size of the centralized database is large, which increases the response time for fetching the data.
- It is not easy to update such an extensive database system.
- If any server failure occurs, entire data will be lost, which could be a huge loss.

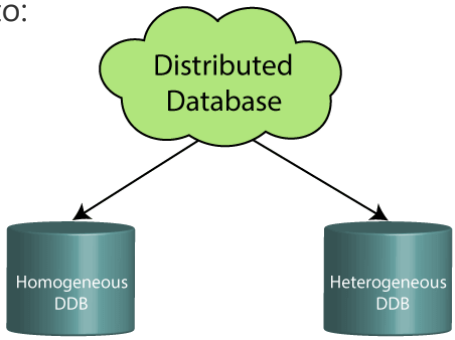
b) Distributed Database

- Data distributed across **multiple locations**. It improves the availability and performance.

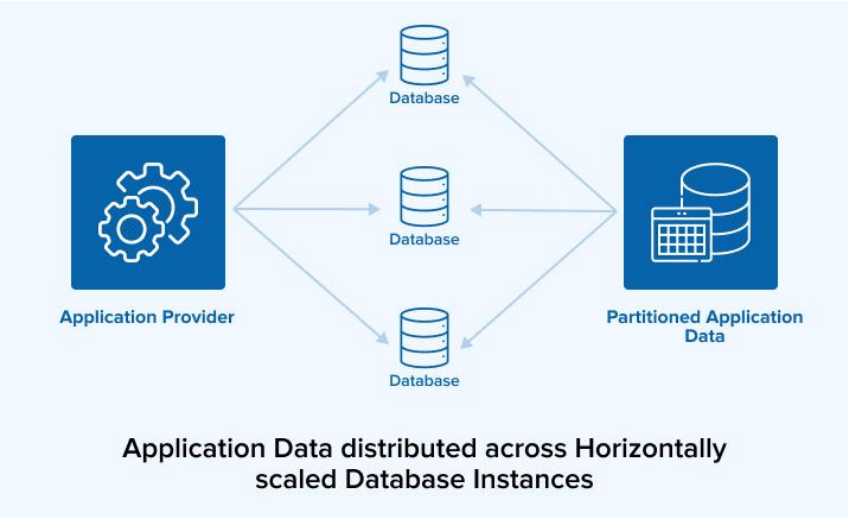


Unlike a centralized database system, in distributed systems, data is distributed among different database systems of an organization. These database systems are connected via communication links. Such links help the end-users to access the data easily. **Examples** of the Distributed database are Apache Cassandra, HBase, Ignite, etc.

We can further divide a distributed database system into:



- **Homogeneous DDB:** Those database systems which execute on the same operating system and use the same application process and carry the same hardware devices.
- **Heterogeneous DDB:** Those database systems which execute on different operating systems under different application procedures, and carries different hardware devices.



Advantages of Distributed Database

- Modular development is possible in a distributed database, i.e., the system can be expanded by including new computers and connecting them to the distributed system.
- One server failure will not affect the entire data set.

c) Cloud Database

- Hosted on **cloud platforms**. Scalable and cost-effective.

A type of database where data is stored in a virtual environment and executes over the cloud computing platform. It provides users with various cloud computing services (SaaS, PaaS, IaaS, etc.) for accessing the database. *Examples:* Amazon RDS, Azure SQL.

d) Data Warehouse

- Stores **historical and analytical data**.
- Used for decision-making.
- *Example:* Amazon Redshift.

3. Based on Purpose

a) Operational Database

- Supports day-to-day transactions.
- The type of database which creates and updates the database in real-time. It is basically designed for executing and handling the daily data operations in several businesses. For example, an organization uses operational databases for managing per day transactions. *Example:* Banking systems.

b) Real-Time Database

- Handles data with **time constraints**. *Example:* Air traffic control systems.

c) Mobile Database

- Used in **mobile applications**. Works with intermittent connectivity.

d) Graph Database

- Focuses on **relationships between data**. *Example:* Social networks (Neo4j).

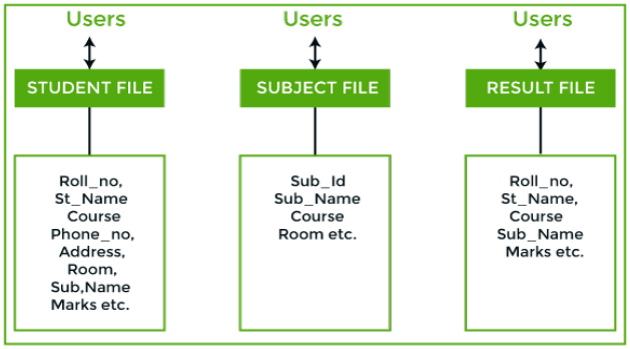
4. Based on Storage

- **In-Memory Database:** Data stored in RAM (fast access)
  Example: Redis
- **Disk-Based Database:** Data stored on disk
  Example: MySQL

DBMS vs. File System

File System Approach

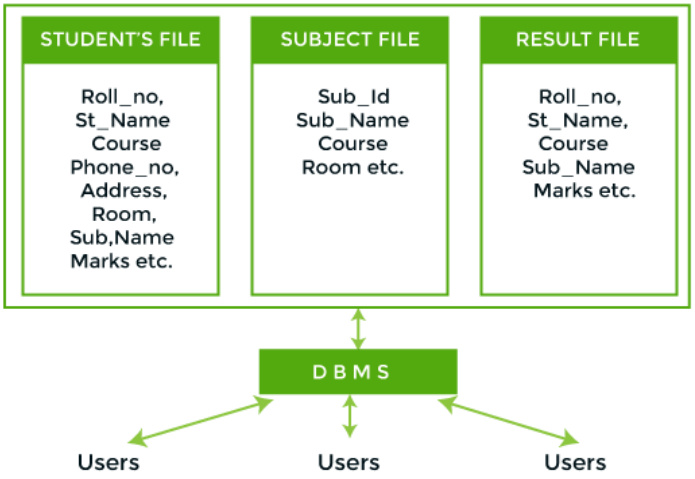
File based systems were an early attempt to computerize the manual system. It is also called a traditional based approach in which a decentralized approach was taken where each department stored and controlled its own data with the help of a data processing specialist. The main role of a data processing specialist was to create the necessary computer file structures, and also manage the data within structures and design some application programs that create reports based on file data.



In the above figure:
Consider an example of a student's file system. The student file will contain information regarding the student (i.e. roll no, student name, course etc.). Similarly, we have a subject file that contains information about the subject and the result file which contains the information regarding the result.
Some fields are duplicated in more than one file, which leads to data redundancy. So to overcome this problem, we need to create a centralized system, i.e. DBMS approach.

DBMS:

A database approach is a well-organized collection of data that are related in a meaningful way which can be accessed by different users but stored only once in a system. The various operations performed by the DBMS system are: Insertion, deletion, selection, sorting etc.



In the above figure, duplication of data is reduced due to centralization of data.

There are the following differences between DBMS and File systems:

Basis	DBMS Approach	File System Approach
Meaning	DBMS is a collection of data. In DBMS, the user is not required to write the procedures.	The file system is a collection of data. In this system, the user has to write the procedures for managing the database.
Sharing of data	Due to the centralized approach, data sharing is easy.	Data is distributed in many files, and it may be of different formats, so it isn't easy to share data.
Data Abstraction	DBMS gives an abstract view of data that hides the details.	The file system provides the detail of the data representation and storage of data.
Security and Protection	DBMS provides a good protection mechanism.	It isn't easy to protect a file under the file system.
Recovery Mechanism	DBMS provides a crash recovery mechanism, i.e., DBMS protects the user from system failure.	The file system doesn't have a crash mechanism, i.e., if the system crashes while entering some data, then the content of the file will be lost.
Manipulation Techniques	DBMS contains a wide variety of sophisticated techniques to store and retrieve the data.	The file system can't efficiently store and retrieve the data.
Concurrency Problems	DBMS takes care of Concurrent access of data using some form of locking.	In the File system, concurrent access has many problems like redirecting the file while deleting some information or updating some information.
Where to use	Database approach used in large systems which interrelate many files.	File system approach used in large systems which interrelate many files.
Cost	The database system is expensive to design.	The file system approach is cheaper to design.
Data Redundancy and Inconsistency	Due to the centralization of the database, the problems of data redundancy and inconsistency are controlled.	In this, the files and application programs are created by different programmers so that there exists a lot of duplication of data which may lead to inconsistency.
Structure	The database structure is complex to design.	The file system approach has a simple structure.

Data Independence	In this system, Data Independence exists, and it can be of two types. ○ Logical Data Independence ○ Physical Data Independence	In the File system approach, there exists no Data Independence.
Integrity Constraints	Integrity Constraints are easy to apply.	Integrity Constraints are difficult to implement in file system.
Data Models	In the database approach, 3 types of data models exist: ○ Hierarchal data models ○ Network data models ○ Relational data models	In the file system approach, there is no concept of data models exists.
Flexibility	Changes are often a necessity to the content of the data stored in any system, and these changes are more easily with a database approach.	The flexibility of the system is less as compared to the DBMS approach.
Examples	Oracle, SQL Server, Sybase etc.	Cobol, C++ etc.

Advantages of DBMS over File System:

Database Management Systems (DBMS) outshine file systems in almost every way.

Data Efficiency:

- **Reduced Redundancy:** DBMS stores data in a structured format (tables), minimizing duplication and saving storage space. Files in a file system often contain redundant information across documents, wasting resources.
- **Centralized Management:** A single system manages all data in a DBMS, making it easier to update, backup, and recover. File systems require

manual management of individual files, leading to inconsistencies and potential data loss.

Data Integrity:

- **Data Validation:** DBMS enforces data constraints (e.g., unique values, data types) to ensure accuracy and consistency. File systems lack inherent validation, leading to errors and invalid data.
- **Transactions:** DBMS guarantees data integrity through ACID transactions (Atomicity, Consistency, Isolation, Durability), ensuring complete and valid data modification even in case of system failures. File systems lack transaction support, making data vulnerable to inconsistencies during updates.

Data Security:

- **User Access Control:** DBMS allows granular access control for different users, protecting sensitive data. File systems provide limited access control mechanisms, making it easier for unauthorized access and modification.
- **Data Encryption:** DBMS offers built-in encryption features to secure sensitive data at rest and in transit. File systems rely on separate encryption tools, adding complexity and potential security vulnerabilities.

Data Sharing and Collaboration:

- **Concurrent Access:** Multiple users can access and modify data simultaneously in a DBMS, facilitating collaboration and efficient data sharing. File systems often lock files during edits, restricting concurrent access and hindering collaboration.
- **Querying and Reporting:** DBMS provides powerful query languages like SQL to search, filter, and analyze data efficiently. File systems require custom scripts or manual searches, making data analysis cumbersome and slow.

Scalability and Flexibility:

- **Easy Growth:** DBMS can readily handle growing data volumes by adding storage or scaling to cloud platforms. File systems become unwieldy with large data sets, requiring complex data migrations and manual expansion.

- **Data Modeling:** DBMS offers various data models (relational, NoSQL) to adapt to different data structures and requirements. File systems lack inherent data models, limiting flexibility and adaptability.

Additional Benefits:

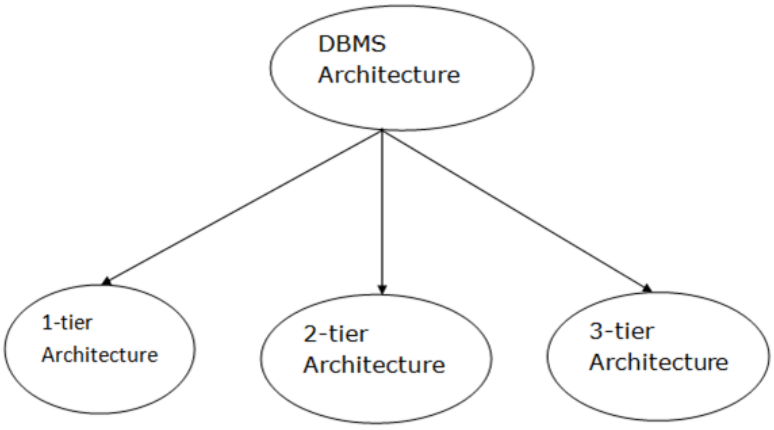
- **Backup and Recovery:** DBMS offers robust backup and recovery mechanisms to minimize data loss. File systems rely on external backup solutions, increasing the risk of data loss from accidental deletions or system failures.
- **Data Analysis Tools:** Many DBMS include built-in reporting and analysis tools, simplifying data exploration and insights generation. File systems require separate tools for data analysis, adding complexity and cost.

While file systems can be simple and familiar, their limitations become evident as data grows and complexity increases. DBMS offers a comprehensive and powerful solution for data management, providing efficiency, security, scalability, and advanced features that file systems simply can't match.

DBMS Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which are connected via the network.
- DBMS architecture depends upon how users are connected to the database to get their request done.

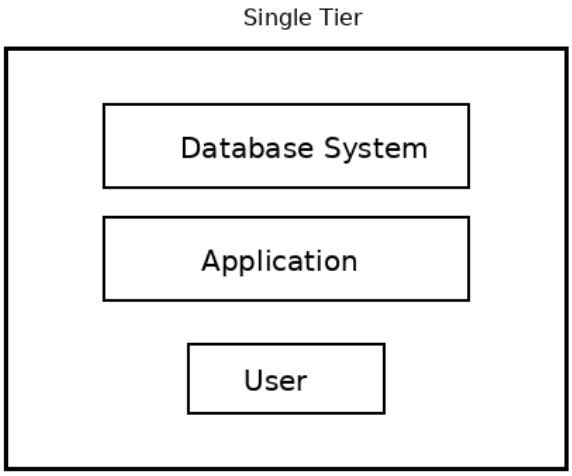
Types of DBMS Architecture



Database architecture can be seen as a single tier or multi-tier. But logically, database architecture is of two types like: **2-tier architecture** and **3-tier architecture**.

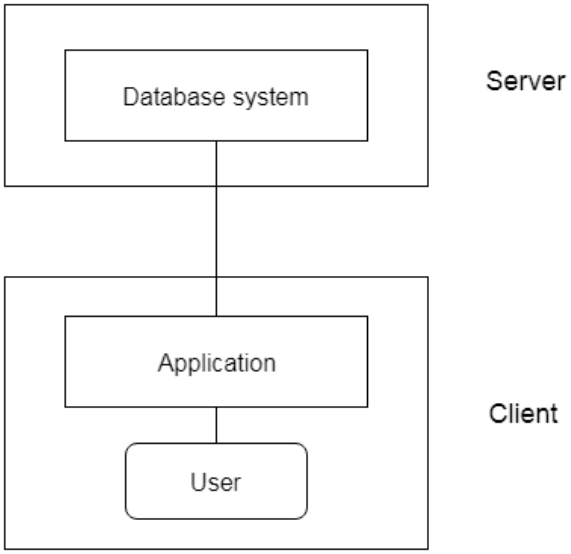
1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.



2-Tier Architecture

- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: **ODBC**, **JDBC** are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.



2-tier Architecture

3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.

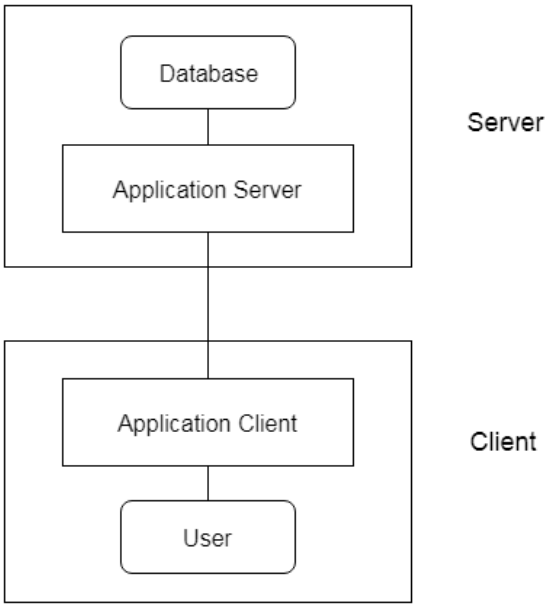
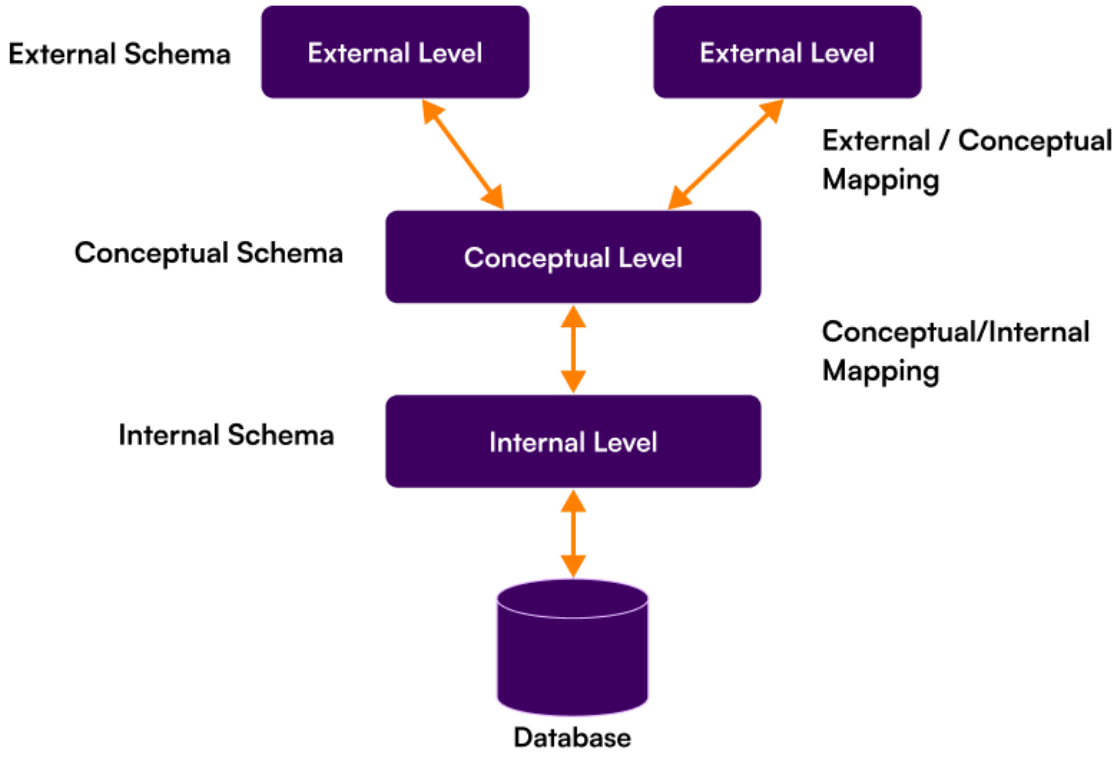


Fig: 3-tier Architecture

Three schema Architecture

- The three-schema architecture is also called ANSI/SPARC architecture or three-level architecture.
- This framework is used to describe the structure of a specific database system.
- It divides a database system into **three levels** to separate user applications from physical data storage.
- This improves **data independence**, security, and flexibility.

The three-schema architecture is as follows:



In the above diagram:

- It shows the DBMS architecture.
- Mapping is used to transform the request and response between various database levels of architecture.
- Mapping is not good for small DBMS because it takes more time.
- In External / Conceptual mapping, it is necessary to transform the request from external level to conceptual schema.
- In Conceptual / Internal mapping, DBMS transform the request from the conceptual to internal level.

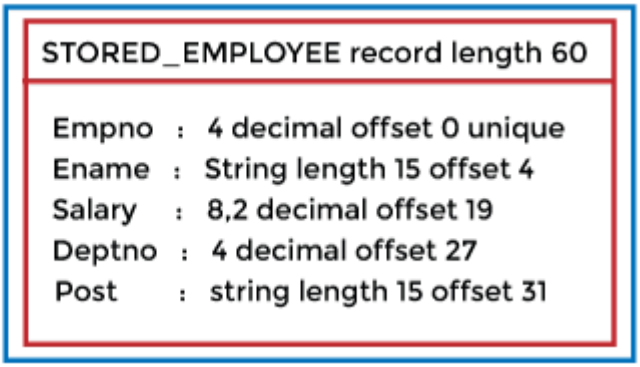
Objectives of Three schema Architecture

The main objective of three level architecture is to enable multiple users to access the same data with a personalized view while storing the underlying data only once. Thus it separates the user's view from the physical structure of the database. This separation is desirable for the following reasons:

- Different users need different views of the same data.
- The approach in which a particular user needs to see the data may change over time.
- The users of the database should not worry about the physical implementation and internal workings of the database such as data compression and encryption techniques, hashing, optimization of the internal structures etc.
- All users should be able to access the same data according to their requirements.
- DBA should be able to change the conceptual structure of the database without affecting the user's
- Internal structure of the database should be unaffected by changes to physical aspects of the storage.

1. Internal Level/ Physical Level

Internal view



- The internal level has an internal schema which describes the physical storage structure of the database.
- The internal schema is also known as a physical schema.
- It uses the physical data model. It is used to define that how the data will be stored in a block.
- The physical level is used to describe complex low-level data structures in detail.
- Defines how data is actually stored on disk.
- Includes file organization, indexing, hashing, and storage paths.
- Concerned with performance optimization.
- Example: B-tree indexes, record placement, data compression.

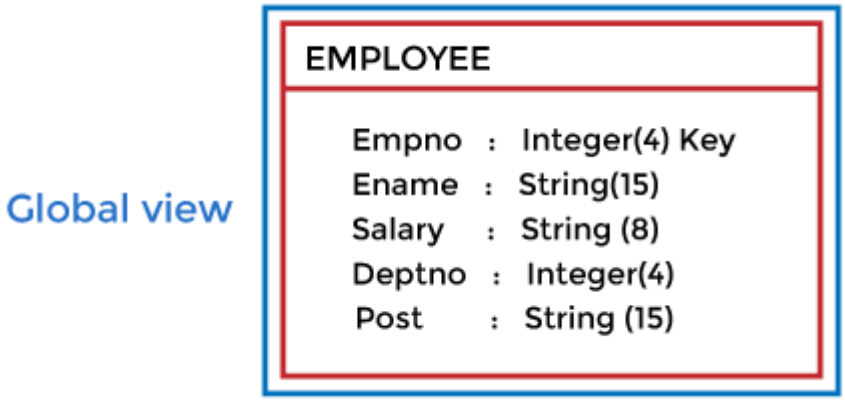
The internal level is generally is concerned with the following activities:

- Storage space allocations.
  For Example: B-Trees, Hashing etc.
- Access paths.

For Example: Specification of primary and secondary keys, indexes, pointers and sequencing.

- Data compression and encryption techniques.
- Optimization of internal structures.
- Representation of stored fields.

2. Conceptual Level/ Logical Level



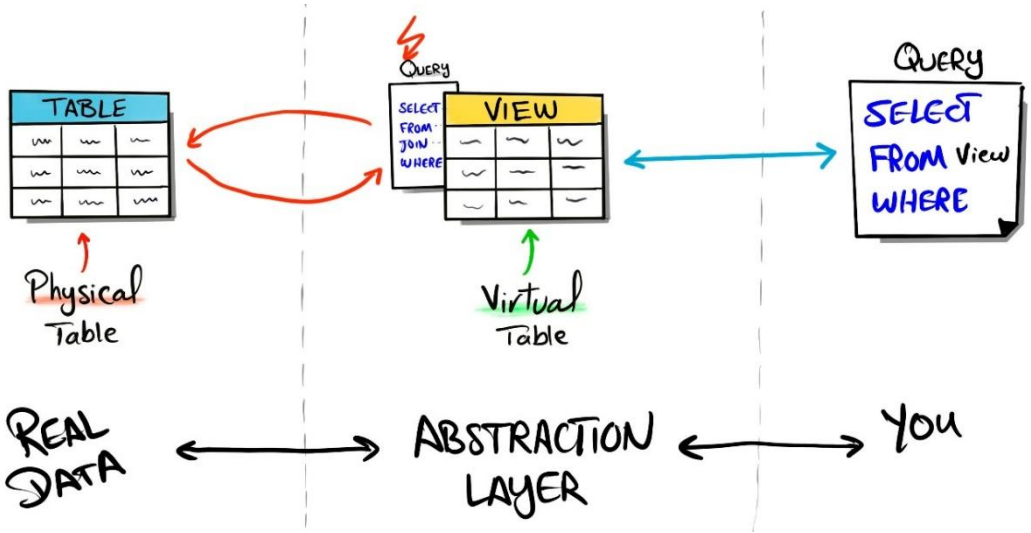
- The conceptual schema describes the design of a database at the conceptual level. Conceptual level is also known as logical level.
- The conceptual schema describes the structure of the whole database.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.
- In the conceptual level, internal details such as an implementation of the data structure are hidden.
- Programmers and database administrators work at this level.
- Complete logical structure of the database. Tables, relationships, constraints
- Describes the entire logical structure of the database.
- Includes entities, attributes, relationships, and constraints.
- Independent of physical storage.
- Example: Tables like Student, Course, Enrollment with relationships.

3. External Level / View Level

- At the external level, a database contains several schemas that sometimes called as subschema. The subschema is used to describe the different view of the database.
- An external schema is also known as view schema.
- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.
- The view schema describes the end user interaction with database systems.
- User-specific views of data. This is where Views belong
  - Represents user-specific views of the database.
  - Each user or application sees only required data.
  - Enhances security and customization.
  - Example: A student sees only marks, while admin sees full student records.
-

Views in DBMS

- A View is a virtual table in a DBMS that is created using a SQL SELECT query.
- It shows only selected data to users.
- It does not store data physically—it shows data derived from one or more base tables.
- Views are implemented at the External Schema level.
- They control **what users see**, not how data is stored



Why Views are Used

- Security: Show only required columns/rows to users
- Simplicity: Hide complex queries
- Consistency: Same logic reused across applications
- Customization: Different users, different views of the same data

Types of Views

1. **Simple View:**
   - Based on a single table
   - No GROUP BY, functions
2. **Complex View**
   - Based on multiple tables
   - Uses joins, aggregates
3. **Updatable View**
   - Allows INSERT/UPDATE (with restrictions)
4. **Read-Only View**
   - Data cannot be modified

View vs Table

Aspect	View	Table
Storage	No physical storage	Physically stored
Nature	Virtual	Real
Data	Derived	Original
Security	High	Depends on permissions

Mapping between Views

The three levels of DBMS architecture don't exist independently of each other. There must be correspondence between the three levels i.e. how they actually correspond with each other. DBMS is responsible for correspondence between the three types of schema. This correspondence is called Mapping.

There are basically two types of mapping in the database architecture:

- Conceptual/ Internal Mapping
- External / Conceptual Mapping

Conceptual/ Internal Mapping

The Conceptual/ Internal Mapping lies between the conceptual level and the internal level. Its role is to define the correspondence between the records and fields of the conceptual level and files and data structures of the internal level.

External/ Conceptual Mapping

The external/Conceptual Mapping lies between the external level and the Conceptual level. Its role is to define the correspondence between a particular external and the conceptual view.